1. Game Character System (Abstract Classes)

```
abstract class Character {
    private String name;
    private int health;
    private int attackPower;

    // Constructor
    public Character(String name) {
        this.name = name;
        this.health = 100;
        this.attackPower = 10;
    }

    // Abstract methods - must be implemented by subclasses
    public abstract void attack();
    public abstract void defend();

    // Concrete method - shared implementation
    public void displayStats() {
        System.out.println("=== Character Stats ===");
        System.out.println("Name: " + name);
        System.out.println("Health: " + health);
        System.out.println("Attack Power: " + attackPower);
    }

    // Getters and setters for encapsulation
    public String getName() {
        return name;
    }

    public int getHealth() {
        return health;
    }

    public void setHealth(int health) {
        this.health = health;
    }

    public int getAttackPower() {
        return attackPower;
    }

    public void setAttackPower(int attackPower) {
        this.attackPower = attackPower;
    }
}

// Warrior subclass
class Warrior extends Character {
    private int armor;

    public Warrior(String name) {
        super(name);
        this.armor = 50;
```

```
        setAttackPower(25);
    }

    @Override
    public void attack() {
        System.out.println(getName() + " swings their sword with mighty force!");
        System.out.println(getName() + " attacks for " + getAttackPower() + " damage!");
    }

    @Override
    public void defend() {
        System.out.println(getName() + " raises their shield, blocking " + armor + " damage!");
    }

    public int getArmor() {
        return armor;
    }
}

// Mage subclass
class Mage extends Character {
    private int mana;

    public Mage(String name) {
        super(name);
        this.mana = 100;
        setAttackPower(35);
    }

    @Override
    public void attack() {
        System.out.println(getName() + " casts a powerful fireball!");
        System.out.println(getName() + " deals " + getAttackPower() + " magical damage!");
    }

    @Override
    public void defend() {
        System.out.println(getName() + " creates a magical barrier!");
    }

    public void castSpell(String spellName) {
        if (mana >= 20) {
            mana -= 20;
            System.out.println(getName() + " casts " + spellName + "! (Mana left: " + mana + ")");
        } else {
            System.out.println(getName() + " doesn't have enough mana!");
        }
    }

    public int getMana() {
        return mana;
    }
}
```

```
// Main class
public class GameCharacterSystem {
    public static void main(String[] args) {
        System.out.println("=== GAME CHARACTER SYSTEM ===\n");

        // Creating Warrior object
        Warrior warrior = new Warrior("Aragorn");
        System.out.println("--- Warrior ---");
        warrior.displayStats();
        warrior.attack();
        warrior.defend();

        System.out.println();

        // Creating Mage object
        Mage mage = new Mage("Gandalf");
        System.out.println("--- Mage ---");
        mage.displayStats();
        mage.attack();
        mage.defend();
        mage.castSpell("Lightning Bolt");
    }
}
```

```
[annkurrrr@archlinux Expt8]$ java GameCharacterSystem
=== GAME CHARACTER SYSTEM ===

--- Warrior ---
=== Character Stats ===
Name: Aragorn
Health: 100
Attack Power: 25
Aragorn swings their sword with mighty force!
Aragorn attacks for 25 damage!
Aragorn raises their shield, blocking 50 damage!

--- Mage ---
=== Character Stats ===
Name: Gandalf
Health: 100
Attack Power: 35
Gandalf casts a powerful fireball!
Gandalf deals 35 magical damage!
Gandalf creates a magical barrier!
Gandalf casts Lightning Bolt! (Mana left: 80)
```

2. Order Processing System (Abstract + Final Methods)

```
abstract class Order {
    private String orderId;
    private double amount;
    private String customerName;
```

```
    private static int orderCounter = 1000;

    // Constructor
    public Order(String customerName, double amount) {
        this.orderId = "ORD-" + (++orderCounter);
        this.customerName = customerName;
        this.amount = amount;
    }

    // Abstract method - must be implemented by subclasses
    public abstract void processOrder();

    // Final method - cannot be overridden by subclasses
    public final void generateInvoice() {
        System.out.println("=== INVOICE ===");
        System.out.println("Order ID: " + orderId);
        System.out.println("Customer: " + customerName);
        System.out.println("Amount: $" + String.format("%.2f", amount));
        System.out.println("Invoice generated successfully!");
        System.out.println("================");
    }

    // Getters and setters
    public String getOrderId() {
        return orderId;
    }

    public double getAmount() {
        return amount;
    }

    public String getCustomerName() {
        return customerName;
    }

    public void setAmount(double amount) {
        this.amount = amount;
    }
}

// OnlineOrder subclass
class OnlineOrder extends Order {
    private String deliveryAddress;
    private String paymentMethod;

    public OnlineOrder(String customerName, double amount, String address, String payment) {
        super(customerName, amount);
        this.deliveryAddress = address;
        this.paymentMethod = payment;
    }

    @Override
    public void processOrder() {
        System.out.println("Processing ONLINE order...");
```

```
        System.out.println("Delivery Address: " + deliveryAddress);
        System.out.println("Payment Method: " + paymentMethod);
        System.out.println("Order will be shipped within 2-3 business days.");
        generateInvoice(); // Calling final method
    }
}

// StoreOrder subclass
class StoreOrder extends Order {
    private String storeLocation;
    private String pickupTime;

    public StoreOrder(String customerName, double amount, String location, String time) {
        super(customerName, amount);
        this.storeLocation = location;
        this.pickupTime = time;
    }

    @Override
    public void processOrder() {
        System.out.println("Processing STORE pickup order...");
        System.out.println("Store Location: " + storeLocation);
        System.out.println("Pickup Time: " + pickupTime);
        System.out.println("Order ready for pickup!");
        generateInvoice(); // Calling final method
    }
}

// Main class
public class OrderProcessingSystem {
    public static void main(String[] args) {
        System.out.println("=== ORDER PROCESSING SYSTEM ===\n");

        // Creating and processing Online Order
        OnlineOrder onlineOrder = new OnlineOrder(
            "John Doe",
            299.99,
            "123 Main St, City, State 12345",
            "Credit Card"
        );
        onlineOrder.processOrder();

        System.out.println();

        // Creating and processing Store Order
        StoreOrder storeOrder = new StoreOrder(
            "Jane Smith",
            149.50,
            "Downtown Store",
            "2:00 PM"
        );
        storeOrder.processOrder();
    }
}
```

```
[annkurrrr@archlinux Expt8]$ java OrderProcessingSystem
=== ORDER PROCESSING SYSTEM ===

Processing ONLINE order...
Delivery Address: 123 Main St, City, State 12345
Payment Method: Credit Card
Order will be shipped within 2-3 business days.
=== INVOICE ===
Order ID: ORD-1001
Customer: John Doe
Amount: $299.99
Invoice generated successfully!
================

Processing STORE pickup order...
Store Location: Downtown Store
Pickup Time: 2:00 PM
Order ready for pickup!
=== INVOICE ===
Order ID: ORD-1002
Customer: Jane Smith
Amount: $149.50
Invoice generated successfully!
================
```

3. Multi-Level Media System (Multilevel Inheritance)

```
abstract class Media {
    private String title;
    private String artist;
    private int duration;

    // Constructor
    public Media(String title, String artist, int duration) {
        this.title = title;
        this.artist = artist;
        this.duration = duration;
    }

    // Abstract method - must be implemented by subclasses
    public abstract void play();

    // Getters and setters
    public String getTitle() {
        return title;
    }

    public String getArtist() {
        return artist;
    }

    public int getDuration() {
        return duration;
    }
```

```
    public void displayInfo() {
        System.out.println("Title: " + title);
        System.out.println("Artist: " + artist);
        System.out.println("Duration: " + duration + " seconds");
    }
}

// Abstract subclass for audio media
abstract class AudioMedia extends Media {
    protected int volume;

    // Constructor
    public AudioMedia(String title, String artist, int duration) {
        super(title, artist, duration);
        this.volume = 50;
    }

    // Abstract method for volume control
    public abstract void adjustVolume(int level);

    // Concrete method for audio-specific functionality
    public void pause() {
        System.out.println(getTitle() + " paused.");
    }

    public void stop() {
        System.out.println(getTitle() + " stopped.");
    }

    public int getVolume() {
        return volume;
    }
}

// Concrete class for music player
class MusicPlayer extends AudioMedia {
    private String genre;
    private boolean isPlaying;

    public MusicPlayer(String title, String artist, int duration, String genre) {
        super(title, artist, duration);
        this.genre = genre;
        this.isPlaying = false;
    }

    @Override
    public void play() {
        isPlaying = true;
        System.out.println("=== Now Playing ===");
        displayInfo();
        System.out.println("Genre: " + genre);
        System.out.println("Volume: " + volume);
        System.out.println("Status: Playing");
```

```
        System.out.println("====================");
    }

    @Override
    public void adjustVolume(int level) {
        if (level >= 0 && level <= 100) {
            this.volume = level;
            System.out.println("Volume adjusted to: " + level);
        } else {
            System.out.println("Invalid volume level! Must be between 0-100.");
        }
    }

    public void togglePlay() {
        if (isPlaying) {
            pause();
            isPlaying = false;
        } else {
            play();
            isPlaying = true;
        }
    }

    public String getGenre() {
        return genre;
    }
}

// Main class
public class MediaSystem {
    public static void main(String[] args) {
        System.out.println("=== MULTI-LEVEL MEDIA SYSTEM ===\n");

        // Creating MusicPlayer object (concrete class)
        MusicPlayer player = new MusicPlayer(
            "Bohemian Rhapsody",
            "Queen",
            354,
            "Rock"
        );

        // Playing media
        player.play();

        System.out.println();

        // Adjusting volume
        player.adjustVolume(75);

        System.out.println();

        // Pausing and resuming
        player.pause();
        player.togglePlay();
```

```
        System.out.println();

        // Stopping
        player.stop();
    }
}
```

```
[annkurrrr@archlinux Expt8]$ java MediaSystem
=== MULTI-LEVEL MEDIA SYSTEM ===

=== Now Playing ===
Title: Bohemian Rhapsody
Artist: Queen
Duration: 354 seconds
Genre: Rock
Volume: 50
Status: Playing
==================

Volume adjusted to: 75

Bohemian Rhapsody paused.
Bohemian Rhapsody paused.

Bohemian Rhapsody stopped.
```

4. Smart Home System (Interface + Abstract Class)

```
interface Automation {
    void autoControl();
}

// Abstract base class for home devices
abstract class HomeDevice {
    private String deviceId;
    private String deviceName;
    private boolean isOn;
    private static int deviceCounter = 0;

    // Constructor
    public HomeDevice(String deviceName) {
        this.deviceId = "DEV-" + (++deviceCounter);
        this.deviceName = deviceName;
        this.isOn = false;
    }

    // Static method - belongs to class, not instance
```

```
    public static String deviceCategory() {
        return "Smart Home Devices";
    }

    // Abstract method - must be implemented by subclasses
    public abstract void operate();

    // Concrete methods
    public void turnOn() {
        isOn = true;
        System.out.println(deviceName + " is now ON");
    }

    public void turnOff() {
        isOn = false;
        System.out.println(deviceName + " is now OFF");
    }

    // Getters and setters
    public String getDeviceId() {
        return deviceId;
    }

    public String getDeviceName() {
        return deviceName;
    }

    public boolean isOn() {
        return isOn;
    }

    public static int getDeviceCounter() {
        return deviceCounter;
    }
}

// SmartLight class extending HomeDevice and implementing Automation
class SmartLight extends HomeDevice implements Automation {
    private int brightness;
    private String color;

    public SmartLight(String deviceName) {
        super(deviceName);
        this.brightness = 50;
        this.color = "White";
    }

    @Override
    public void operate() {
        if (isOn()) {
            System.out.println(getDeviceName() + " is operating at " + brightness + "% brightness");
            System.out.println("Color: " + color);
        } else {
            System.out.println(getDeviceName() + " needs to be turned ON first!");
```

```
        }
    }

    @Override
    public void autoControl() {
        System.out.println("=== Auto Control Mode ===");
        if (isOn()) {
            // Simulate automatic brightness adjustment
            brightness = 70;
            color = "Warm Yellow";
            System.out.println("Automatically adjusted brightness to " + brightness + "%");
            System.out.println("Changed color to " + color);
        } else {
            turnOn();
            brightness = 50;
            System.out.println("Auto-turned on at " + brightness + "% brightness");
        }
    }

    public void setBrightness(int brightness) {
        if (brightness >= 0 && brightness <= 100) {
            this.brightness = brightness;
            System.out.println("Brightness set to: " + brightness + "%");
        }
    }

    public void setColor(String color) {
        this.color = color;
        System.out.println("Color changed to: " + color);
    }

    public int getBrightness() {
        return brightness;
    }

    public String getColor() {
        return color;
    }
}

// Additional class: SmartThermostat for demonstration
class SmartThermostat extends HomeDevice implements Automation {
    private int temperature;

    public SmartThermostat(String deviceName) {
        super(deviceName);
        this.temperature = 72;
    }

    @Override
    public void operate() {
        if (isOn()) {
            System.out.println(getDeviceName() + " maintaining " + temperature + "F");
        } else {
```

```
            System.out.println(getDeviceName() + " is currently OFF");
        }
    }

    @Override
    public void autoControl() {
        System.out.println("=== Auto Control Mode ===");
        // Simulate automatic temperature adjustment
        temperature = 68;
        System.out.println("Automatically adjusted temperature to " + temperature + "F (Energy Saving Mode)");
    }

    public void setTemperature(int temp) {
        this.temperature = temp;
        System.out.println("Temperature set to: " + temperature + "F");
    }
}

// Main class
public class SmartHomeSystem {
    public static void main(String[] args) {
        System.out.println("=== SMART HOME SYSTEM ===\n");

        // Display device category using static method
        System.out.println("Category: " + HomeDevice.deviceCategory());
        System.out.println();

        // Creating SmartLight object
        SmartLight livingRoomLight = new SmartLight("Living Room Light");

        System.out.println("--- Smart Light Control ---");
        System.out.println("Device ID: " + livingRoomLight.getDeviceId());
        System.out.println("Device Name: " + livingRoomLight.getDeviceName());

        // Manual control
        livingRoomLight.turnOn();
        livingRoomLight.operate();

        System.out.println();

        // Auto control via interface
        livingRoomLight.autoControl();

        System.out.println();

        // Additional settings
        livingRoomLight.setBrightness(80);
        livingRoomLight.setColor("Blue");
        livingRoomLight.operate();

        System.out.println();
        System.out.println("--- Smart Thermostat Control ---");

        // Creating SmartThermostat object
```

```
        SmartThermostat thermostat = new SmartThermostat("Main Thermostat");
        thermostat.turnOn();
        thermostat.operate();
        thermostat.autoControl();

        System.out.println();
        System.out.println("Total devices created: " + HomeDevice.getDeviceCounter());
    }
}
```

```
[annkurrrr@archlinux Expt8]$ java SmartHomeSystem
=== SMART HOME SYSTEM ===

Category: Smart Home Devices

--- Smart Light Control ---
Device ID: DEV-1
Device Name: Living Room Light
Living Room Light is now ON
Living Room Light is operating at 50% brightness
Color: White

=== Auto Control Mode ===
Automatically adjusted brightness to 70%
Changed color to Warm Yellow

Brightness set to: 80%
Color changed to: Blue
Living Room Light is operating at 80% brightness
Color: Blue

--- Smart Thermostat Control ---
Main Thermostat is now ON
Main Thermostat maintaining 72F
=== Auto Control Mode ===
Automatically adjusted temperature to 68F (Energy Saving

Total devices created: 2
```